191.002 VU Betriebssysteme

EXERCISE 2

Last update 2026-10-29 (Version 6f3271b3)

quadtree

Implement a program that executes spatial queries on 2D point data.

```
SYNOPSIS
    quadtree <goal> <distance> <min_x> <max_x> <min_y> <max_y>
```

Instructions

The program takes multiple lines as input and looks for points that are closer than or equal `distance` to `goal` using a recursive search variant based on quadtrees[1]. The input is read from `stdin` and ends when an `EOF` (End Of File) is encountered.

Quadtrees work similar to binary trees by splitting their 2D search space into four quadrants based on a pivot element or value. In this exercise you may assume that any point is inside the rectangle defined by `min_x`, `max_x`, `min_y` and `max_y` and may choose the center of the rectangle as pivot. Refer to Fig. 1 for axis orientations and naming of the quadrants in this assignment.

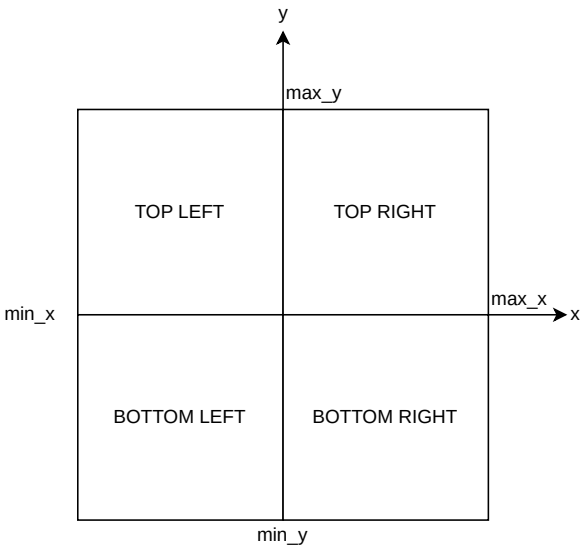


Figure 1: Scheme for the limits in the quadtree.

Your program must accept any number of lines.

The program searches the space recursively, i.e., by calling itself:

1. Parse the goal position, maximum distance and the limits of the search area using *strtof(3)*. The goal position uses the format `<x>,<y>`, make sure to check for the comma between the two components. Exit with status `EXIT_FAILURE` if any of the values fail to parse.

[1] `https://en.wikipedia.org/wiki/Quadtree`

2. If the input consist of only 1 line, then check if it is close enough to the goal point (*powf(3)*, *sqrtf(3)*), write it to `stdout`, starting with the distance using three decimals, a colon and then the point, and exit with status `EXIT_SUCCESS`.

3. Otherwise the input consist of $n > 1$ lines. Ignore all lines that are equal to the first line. If all lines are the same, exit with status `EXIT_SUCCESS`.

4. Using *fork(2)* and *execlp(3)*, recursively execute this program in four child processes with the remaining lines. Use the argument `arg` of *execlp(3)* to pass the correct limits to each of the child processes. Use two unnamed pipes per child to redirect *stdin* and *stdout* (see *pipe(2)* and *dup2(3)*). Each line consists of one point. Parse them and split them into according to the pivot element, using one child for each quadrant. The four child processes must run simultaneously!

5. Use *wait(2)* or *waitpid(2)* to read the exit status of the children. Terminate the program with exit status `EXIT_FAILURE` if the exit status of any of the child processes is not `EXIT_SUCCESS`.

6. Merge the sorted parts from the child processes and write them to *stdout*. Start with the output of the bottom-left quadrant and print each line to *stdout*, continue with bottom-right, top-left and top-right. This allows for consistent output, which should help during testing. Terminate the program with exit status `EXIT_SUCCESS`.

Hints

- Add `-lm` to your linker invocation if you are using `powf` and `sqrtf`.
- In order to avoid endless recursion[2], fork only if the input number is greater than 1.
- Quadtrees can grow very deep when values are close together. When using the geometric center as pivot value, use integer values for testing to avoid deep recursion.
- To output error messages and debug messages, always use *stderr* because *stdout* is redirected in most cases.
- You can use the `sort` program to sort your input for debugging purposes.

Testing

Input is shown in blue color. Output to *stdout* (and *stderr*) is shown in black. (Note that in the following output sections `EXIT_SUCCESS` equals `0`, and `EXIT_FAILURE` equals `1`. Refer to `stdlib.h` for further details.) `^C` indicates `CTRL+C`, `^D` indicates `CTRL+D`. The placeholder `<usage message>` must be replaced by a proper usage message (printed to *stdout*), `<error message>` must be replaced by a meaningful error message (which is printed to *stderr*).

Test your program with various inputs, such as:

```
$ cat 1.txt
1,1
2,-2
-3,3
-4,-4
$ ./quadtree 0,0 4.5 -10 10 -10 10 < 1.txt
2.828: 2,-2
4.243: -3,3
1.414: 1,1
$ ./quadtree 0,0 4.5 -10 10 -10 10 < 1.txt | sort  # sorted output
1.414: 1,1
2.828: 2,-2
4.243: -3,3
```

[2] http://en.wikipedia.org/wiki/Fork_bomb

Testcases

Testcase 01: one-line

```
$ echo -e '1,1' | ./quadtree 2,2 1.5 -10 10 -10 10
1.414: 1,1
$ echo $?
0
```

Testcase 02: easy-1

```
$ echo -e "-10,10\n12,10\n2,3\n4,4" | ./quadtree 2,2 7 -15 15 -15 15
1.000: 2,3
2.828: 4,4
$ echo $?
0
```

Testcase 03: easy-2

```
$ echo -e "1,1\n0,1\n1,0\n2,0\n2,1\n2,2\n1,2\n2,0\n2,3,0\n3,1\n3,2\n3,3\n2,3\n1,3\n0,3" | ./quadtree 0,0 \
    3.1 0 4 0 4
1.000: 1,0
1.000: 0,1
1.414: 1,1
2.000: 2,0
2.236: 2,1
2.000: 0,2
2.236: 1,2
2.828: 2,2
3.000: 3,0
3.000: 0,3
$ echo $?
0
```

Testcase 04: easy-3

```
$ echo -e "-10,-10\n10,10\n-20,20\n-10,10\n20,20\n-20,-20\n10,-10\n20,-20" | ./quadtree 0,0 15 -30 30 \
    -30 30
14.142: -10,-10
14.142: 10,-10
14.142: -10,10
14.142: 10,10
$ echo $?
0
```

Testcase 05: 10points

```
$ ( for i in `seq 1 10`; do echo "points_$i" | md5sum | cut -d' ' -f1 | awk -M 'BEGIN { OFS="," } { \
    print strtonum("0x" $1) % 50, (strtonum("0x" $1) * 17) % 50 }'; done; ) > testfile10
$ md5sum testfile10
818cfe2ff5fc17838554ddbdaa33fc9b  testfile10
$ ./quadtree 26,27 12 0 50 0 50 < testfile10
8.062: 19,23
11.180: 37,29
$ echo $?
0
```

Testcase 06: 100points

```
$ ( for i in `seq 1 100`; do echo "foobar_$i" | md5sum | cut -d' ' -f1 | awk -M 'BEGIN { OFS="," } \
    {print strtonum("0x" $1) % 100, (strtonum("0x" $1) * 17) % 100}'; done; ) > testfile100
$ md5sum testfile100
8c5485ce0c511fd81e6c648fed8c54ef  testfile100
$ ./quadtree 50,50 17 0 100 0 100 < testfile100
6.325: 44,48
0.000: 50,50
15.811: 55,35
15.811: 45,65
6.325: 56,52
12.649: 62,54
$ echo $?
0
```

Testcase 07: large-area

```
$ ( for i in `seq 1 100`; do echo "osue_$i" | md5sum | cut -d' ' -f1 | awk -M 'BEGIN { OFS="," } {print \
    strtonum("0x" $1) % 10000, (strtonum("0x" $1) * 17) % 10000}'; done; ) > testfile100_large
$ md5sum testfile100_large
ccff89ef373a0c6c14bc48d052296afe  testfile100_large
$ ./quadtree 1300,1300 700 0 10000 0 10000 < testfile100_large
595.349: 1854,1518
325.208: 1272,1624
$ echo $?
0
```

Testcase 08: invalid-input-1

```
$ ./quadtree 10
<usage message>
$ echo $?
1
```

Testcase 09: invalid-input-2

```
$ echo -e '1,' | ./quadtree 2,2 1.5 -10 10 -10 10
<error message>
$ echo $?
1
```

Additional testcases

Input is shown in blue color. Output to *stdout* (and *stderr*) is shown in black. (Note that in the following output sections `EXIT_SUCCESS` equals `0`, and `EXIT_FAILURE` equals `1`. Refer to `stdlib.h` for further details.) `^C` indicates `CTRL+C`, `^D` indicates `CTRL+D`. The placeholder `<usage message>` must be replaced by a proper usage message (printed to *stdout*), `<error message>` must be replaced by a meaningful error message (which is printed to *stderr*). These testcases are not executed automatically in the submission system, but should be no problem for your implementation.

Make sure to test those testcases only when you are sure your implementation works with duplicated entries.

Testcase: additional-1

```
$ echo -n "1,1\n1,1" > ./quadtree 0,0 1.5 0 5 0 5
1.414: 1,1
$ echo $?
0
```

Testcase: additional-2

```
$ echo -n "1,1\n2,2\n1,1" > ./quadtree 0,0 1.5 0 5 0 5
1.414: 1,1
2.828: 2,2
$ echo $?
0
```

Testcase: additional-2500points

```
$ ( for i in `seq 1 2500`; do echo "test_$i" | md5sum | cut -d' ' -f1 | awk -M 'BEGIN { OFS="," } {print \
    strtonum("0x" $1) % 1000, (strtonum("0x" $1) * 17) % 1000}'; done; ) > testfile2500
$ md5sum testfile2500
1b63065c7450ae72fd28fa145fb734e5  testfile2500
$ ./quadtree 720,450 54 0 1000 0 1000 < testfile2500
41.231: 730,410
25.495: 731,427
47.854: 673,441
46.690: 674,458
51.478: 675,475
13.416: 732,444
17.029: 733,461
31.305: 734,478
47.434: 735,495
```

Testcase: additional-10000points

```
$ ( for i in `seq 1 10000`; do echo "points_are_fun_$i" | md5sum | cut -d' ' -f1 | awk -M 'BEGIN { \
    OFS="","" } {print strtonum("0x" $1) % 10000, (strtonum("0x" $1) * 17) % 10000}'; done; ) > \
    testfile10000
$ md5sum testfile10000
df6c95ad5dd9b019c57129aa185eb76f  testfile10000
$ ./quadtree 9800,9800 200 0 10000 0 10000 < testfile10000
193.003: 9985,9745
189.842: 9986,9762
188.175: 9987,9779
189.447: 9989,9813
196.698: 9991,9847
```

Recommendations and Guidelines

In order to make all solutions more uniform and to avoid problems with different C dialects, please try to follow the following guidelines.

General Guidelines

Try to follow the following guidelines closely.

1. All source files of your program(s) should compile with
   ```
   $ gcc -std=c99 -pedantic -Wall -D_DEFAULT_SOURCE -D_BSD_SOURCE -D_SVID_SOURCE
                                  -D_POSIX_C_SOURCE=200809L -g -c filename.c
   ```
   without *warnings and info messages* and your program(s) should link without warnings too.
2. There should be a Makefile implementing the targets: `all` to build the program(s) (i.e. generate executables) from the sources (this must be the first target in the Makefile); `clean` to delete all files that can be built from your sources with the Makefile.
3. Make all targets of your Makefile idempotent, i.e. execution of `make clean; make clean` should yield the same result as `make clean`, and should not fail with an error.
4. The program shall operate according to the specification/assignment without major issues (e.g., segmentation fault, memory corruption).
5. Arguments should be parsed according to UNIX conventions (we strongly encourage the use of `getopt(3)`). The program should conform to the given synopsis/usage in the assignment. If the synopsis is violated (e.g., unspecified options or too many arguments), the program should terminate with the usage message containing the program name and the correct calling syntax. Argument handling should also be implemented for programs without arguments.
6. Correct (=normal) termination should include a cleanup of resources.
7. Upon success the program shall terminate with exit code 0, in case of errors with an exit code greater than 0. We recommend to use the macros `EXIT_SUCCESS` and `EXIT_FAILURE` (defined in `stdlib.h`) to enable portability of the program.
8. If a function indicates an error with its return value, it *should* be checked in general. If the subsequent code depends on the successful execution of a function (e.g., resource allocation), it is very *important* to check the return value.
9. Functions that do not take any parameters should be declared with `void` in the signature, e.g., `int get_random_int(void);`.
10. Procedures (i.e., functions that do not return a value) should be declared as `void`.
11. Error messages shall be written to `stderr` and should contain the program name `argv[0]`.
12. Do not use the following functions to avoid crashes due to invalid inputs:
    - `gets(3)`; use `fgets(3)` instead.
    - `scanf(3)` and `fscanf(3)`; use `fgets(3)` and `sscanf(3)` instead.
    - `atoi(3)` and `atol(3)`; use `strtol(3)` instead.
13. Documentation is a nice bonus. Format the documentation in Doxygen style (see Wiki and Doxygen's intro).

14. Write meaningful comments. For example, meaningful comments describe the algorithm, or why a particular solution has been chosen, if there seems to be an easier solution at a first glance. Avoid comments that just repeat the code itself
(e.g., `i = i + 1; /* i is incremented by one */`).

15. The documentation of a module should include: name of the module, name and student id of the author (`@author` tag), purpose of the module (`@brief`, `@details` tags) and creation date of the module (`@date` tag).

    The Makefile should also include a header, with author and program name at least.

16. Each function shall be documented either before the declaration or the implementation. It should include purpose (`@brief`, `@details` tags), description of parameters and return value (`@param`, `@return` tags) and description of global variables the function uses (`@details` tag).

    You should also document `static` functions (see `EXTRACT_STATIC` in the file `Doxyfile`). Document visible/exported functions in the header file and local (`static`) functions in the C file. Document variables, constants and types (especially `structs`) too.

17. Documentation, names of variables and constants is best done in English.

18. Internal functions shall be marked with the `static` qualifier and are not allowed to be exported (e.g., in a header file). Only functions that are used by other modules shall be declared in the header file.

19. All exercises shall be solved with functions of the C standard library. If a required function is not available in the standard library, you can use other (external) functions too. Avoid reinventing the wheel (e.g., re-implementation of `strcmp`).

20. Name of constants shall be written in upper case, names of variables in lower case (possibly with the first letter capitalized).

21. Use meaningful variable and constant names (e.g., also semaphores and shared memories).

22. Avoid using global variables as far as possible.

23. All boundaries shall be defined as constants (macros). Avoid arbitrary boundaries. If boundaries are necessary, treat its crossing.

24. Avoid side effects with `&&` and `||`, e.g., write `if(b != 0) c = a/b;` instead of `if(b != 0 && c = a/b)`.

25. Each `switch` block should contain a `default` case. If the case is not reachable, write `assert(0)` to this case (defensive programming).

26. Logical values shall be treated with logical operators, numerical values with arithmetic operators (e.g., test 2 strings for equality by `strcmp(...) == 0` instead of `!strcmp(...)`).

27. Indent your source code consistently (there are tools for that purpose, e.g., `indent`).

28. Avoid tricky arithmetic statements. Programs are written once, but read many times. Your program is not always better if it is shorter!

29. For all I/O operations (read/write from/to `stdin`, `stdout`, files, sockets, pipes, etc.) use *either* standard I/O functions (`fdopen(3)`, `fopen(3)`, `fgets(3)`, etc.) *or* POSIX functions (`open(2)`, `read(2)`, `write(2)`, etc.). Remember, standard I/O functions are buffered. Mixing standard I/O functions and POSIX functions to access a common file descriptor can lead to undefined behaviour and is therefore a bad idea.

30. If asked in the assignment, you should implement signal handling (`SIGINT`, `SIGTERM`). You must use only *async-signal-safe* functions in your signal handlers.

31. Close files, free dynamically allocated memory, and remove resources after usage.

32. Don't waste resources due to inconvenient programming. Header files shall not include implementation parts (exception: macros).

33. To comply with the given testcases, the program output must exactly match the given specification. Therefore you should only print debug information if the compile flag `-DDEBUG` is set, in order to avoid problems with the automatic checker.

Exercise 2 Guidelines

The following points are related for Exercise 2.

1. Correctly use fork/exec/pipes as shown in the lectures. For example, do not exploit inherited memory areas.

2. Ensure termination of child processes without `kill(2)` or `killpg(2)`. Collect the exit codes of child processes (`wait(2)`, `waitpid(2)`, `wait3(2)`).